

Identificación de Vulnerabilidad Crítica: Inyección SQL en el módulo User ID de DVWA

2. Introducción

Este reporte se ha elaborado intentando seguir los lineamientos de la norma **ISO 27001**, en lo que respecta a la gestión y registro de incidentes de seguridad de la información. El objetivo es documentar el hallazgo de una debilidad en la aplicación web **DVWA (Damn Vulnerable Web Application)** en un entorno controlado. La detección temprana y el reporte formal de estas vulnerabilidades permiten reducir riesgos antes de que sean explotados por atacantes, garantizando así la confidencialidad e integridad de los datos.

3. Descripción del Incidente

Durante la revisión del módulo de seguridad de la aplicación, se detectó que el formulario de entrada de texto (diseñado para buscar IDs de usuarios) no valida ni filtra correctamente la información que introduce el usuario.

Al no sanitizar los caracteres especiales (como las comillas simples ' '), la aplicación confunde el texto introducido con código de programación. Esto permite que un usuario modifique la consulta original que se le envía a la base de datos, logrando que el sistema devuelva más información de la que debería.

4. Proceso de Reproducción

Para demostrar la existencia de la vulnerabilidad, se siguieron los siguientes pasos:

1. Se ingresó a la aplicación **DVWA** y se seleccionó el nivel de seguridad **"Low"** (Bajo).
2. Se navegó hasta la sección **"SQL Injection"**.
3. En el campo de texto donde se solicita el **User ID**, en lugar de un número, se introdujo el siguiente código (payload):
4. `1' OR '1'='1'`
5. Se presionó el botón **"Submit"**.

¿Qué pasó en la base de datos? La aplicación web tenía una consulta interna parecida a esta: `SELECT first_name, last_name FROM users WHERE user_id = '$id';`

Al meter nuestro código, la consulta se transformó en: `SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';`

Como la condición `'1'='1'` siempre es verdadera (siempre es "true"), la base de datos se saltó la restricción del ID y mostró los nombres, apellidos y datos de **todos** los usuarios registrados en el sistema.

5. Impacto del Incidente

Este incidente tiene un impacto **Crítico** sobre la **Confidencialidad** de la información:

- **Fuga de Datos:** Un atacante puede ver información de cuentas ajenas (nombres, IDs, y potencialmente contraseñas o datos personales).
- **Riesgo de Escalabilidad:** Un atacante no solo podría leer datos, sino también modificarlos, borrarlos o incluso tomar el control total del servidor web.

6. Recomendaciones

Para solucionar esta vulnerabilidad se recomienda:

- **Uso de Consultas Preparadas:** Separar los datos que introduce el usuario del código de la consulta SQL. De esta forma, la base de datos siempre tratará el texto `1' OR '1'='1` como un simple nombre o texto, y no como una orden ejecutable.
- **Validación de Entradas:** Configurar el formulario para que solo acepte números enteros (ya que es un campo de ID), rechazando inmediatamente cualquier comilla o palabra clave de SQL.
- **Principio de Menor Privilegio:** Asegurarse de que el usuario de la base de datos que usa la web solo tenga permisos para hacer lo estrictamente necesario (por ejemplo, leer y no borrar tablas).

7. Conclusiones

La identificación de esta inyección SQL demuestra que las aplicaciones que interactúan con bases de datos no pueden confiar ciegamente en lo que los usuarios escriben en las pantallas. La gestión de incidentes es un proceso de **mejora continua**. Corregir esta vulnerabilidad de forma protegerá el activo más valioso de la organización: sus datos.